

Traitement de la réponse OCR et persistance en base de données

Fichier source : [src/routes/ocr.py](#)

Le bloc de code ci-dessous détaille la logique métier exécutée par l'API FastAPI une fois que le service d'Intelligence Artificielle externe (OCR.space) a analysé le document. Ce processus inclut :

- la gestion de la dégradation gracieuse en cas d'erreur de traitement
- le nettoyage et l'extraction des données par expressions régulières
- l'insertion sécurisée (requêtes paramétrées) dans la base de données PostgreSQL.
- Observabilité (MLOps) : Avec `logger` et surtout l'incrémentation de la métrique Prometheus `OCR_FAILURES.inc()`.
- Gestion des erreurs : Le renvoi du statut pending sans faire planter l'API.
- Parsing de données complexes : Les expressions régulières appliquées au texte brut de l'IA.
- Cybersécurité et persistance SQL : L'utilisation de requêtes paramétrées (%) avec `psycopg2` qui protègent la base PostgreSQL contre les injections SQL.

```
# 4. Analyse et parsing de la réponse d'OCR.space
# Log de la réponse brute pour diagnostiquer les erreurs silencieuses
logger.debug("ocr_raw_response", extra={"response": resultat_json})

if resultat_json.get("IsErroredOnProcessing", False):
    error_msg = resultat_json.get("ErrorMessage", ["Erreur inconnue"])
    logger.error(
        "ocr_processing_error",
        extra={
            "client_id": client_id,
            "error": str(error_msg),
            "exit_code": resultat_json.get("OCRExitCode"),
        }
    )
    # Incrémentation de la métrique Prometheus pour la supervision (Alerte Grafana)
    OCR_FAILURES.inc()
    return {
        "status": "pending",
        "message": "Le service OCR a rencontré une erreur de traitement (quota ou
        format non supporté). Votre fichier est mis en file d'attente."
    }

# Extraction du texte brut fusionné de toutes les pages
parsed_results = resultat_json.get("ParsedResults", [])
texte_brut_extrairet: str = ""
for page in parsed_results:
    texte_brut_extrairet += page.get("ParsedText", "")
```

```

# 5. Extraction structurée des paramètres physico-chimiques (Regex)
# L'algorithme cherche par exemple "pH : 7.4" ou "ph=7.4" dans le texte brut
mesures: Dict[str, float] = {
    "ph": _extraire_valeur_metroique(texte_brut_extraite, r"ph[:\s\s=]+([0-9.]+)",
        7.2),
    "Hardness": _extraire_valeur_metroique(texte_brut_extraite, r"durete[:\s\s=]+
        ([0-9.]+)", 200.0),
    "Solids": _extraire_valeur_metroique(texte_brut_extraite, r"solides[:\s\s=]+
        ([0-9.]+)", 20000.0),
    "Chloramines": _extraire_valeur_metroique(texte_brut_extraite, r"chloramines[:
        \s\s=]+([0-9.]+)", 3.5),
    "Sulfate": _extraire_valeur_metroique(texte_brut_extraite, r"sulfate[:\s\s=]+
        ([0-9.]+)", 330.0),
    "Conductivity": _extraire_valeur_metroique(texte_brut_extraite, r"conductivite[:
        \s\s=]+([0-9.]+)", 420.0),
    "Organic_carbon": _extraire_valeur_metroique(texte_brut_extraite, r"carbone[:
        \s\s=]+([0-9.]+)", 14.0),
    "Trihalomethanes": _extraire_valeur_metroique(texte_brut_extraite,
        r"trihalomethanes[:\s\s=]+([0-9.]+)", 65.0),
    "Turbidity": _extraire_valeur_metroique(texte_brut_extraite, r"turbidite[:\s\s=]+
        ([0-9.]+)", 3.8)
}

# 6. Persistence dans PostgreSQL sous la provenance "OCR"
query_insert: str = """
INSERT INTO prelevements (
    client_id, provenance, ph, hardness, solids, chloramines,
    sulfate, conductivity, organic_carbon, trihalomethanes,
    turbidity, observations
) VALUES (%s, 'OCR', %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
RETURNING id;
"""

try:
    # Ouverture de la connexion à la BDD via psycopg2
    with psycopg2.connect(settings.DATABASE_URL) as conn:
        with conn.cursor() as cursor:
            # Exécution sécurisée pour prévenir les injections SQL
            cursor.execute(query_insert, (
                client_id, mesures["ph"], mesures["Hardness"], mesures["Solids"],
                mesures["Chloramines"], mesures["Sulfate"],
                mesures["Conductivity"],
                mesures["Organic_carbon"], mesures["Trihalomethanes"],
                mesures["Turbidity"], f"Fichier d'origine : {nom_fichier}"
            ))
            row = cursor.fetchone()
            if row is None:
                raise Exception("INSERT n'a retourné aucun ID de prélèvement")
            prelevement_id: int = row
            conn.commit()

    return {
        "status": "Succès",
        "message": "La fiche laboratoire a été numérisée et enregistrée.",
        "prelevement_id": prelevement_id,
        "extracted_data": mesures
    }

```